Solving the Minimum Variance Portfolio Fast: A Krylov Perspective

Thomas Schmelzer* Michael Wolf

Abstract

The long-only minimum variance portfolio is typically solved by forming a sample covariance matrix and passing it to a generic quadratic programming solver. This standard approach obscures the structure of the problem and incurs unnecessary computational and memory costs.

We show that the problem is more naturally expressed as a matrix-free least-squares problem on the returns matrix itself. This reformulation eliminates the need to form the covariance matrix and exposes a symmetric indefinite KKT system that can be solved efficiently using Krylov subspace methods. Both MINRES and a constraint-eliminated conjugate gradient method outperform direct solves and generic optimisation routines by large factors. On real equity returns, where factor structure inflates the condition number, Ledoit-Wolf shrinkage reduces iteration counts substantially; with shrinkage applied the two Krylov methods are competitive, with CG marginally faster.

A central insight of the paper is that covariance shrinkage, as introduced by Ledoit and Wolf, acts as a spectral preconditioner for the underlying linear system. This establishes a direct link between statistical estimation and numerical linear algebra: both operate by reshaping the eigenvalue spectrum of the same operator. Empirically, shrinkage significantly reduces Krylov iteration counts, providing computational gains alongside its statistical benefits.

The resulting perspective recasts minimum variance portfolio construction as a structured linear algebra problem rather than a generic optimisation task, enabling scalable solvers that are simple, fast, and well suited to large asset universes.

1 Introduction

The minimum variance portfolio is one of the most fundamental constructions in quantitative finance. Since [7], it has been standard practice to estimate a covariance matrix from historical returns and solve the resulting quadratic program using a general-purpose optimisation routine.

This workflow is convenient but conceptually misaligned with the structure of the problem. The covariance matrix is not a primitive object; it is an estimate derived from a returns matrix. Once formed, this estimate obscures the underlying data structure and forces the use of dense linear algebra, even though all required operations can be expressed in terms of matrix-vector products on the returns themselves.

In this paper we take a different view. We show that the minimum variance problem is more naturally formulated as a least-squares problem on the returns matrix, avoiding the explicit construction of the covariance matrix altogether. This seemingly minor change has significant consequences. It exposes the problem as a structured linear system — specifically, a symmetric

*The first author's 2004 diploma thesis at the University of Kaiserslautern [9], supervised by Martin H. Gutknecht (ETH Zürich), developed block extensions of MINRES and SYMMLQ. The present work revisits those same methods in a new setting twenty-two years later.

indefinite KKT system — that can be solved efficiently using Krylov subspace methods such as MINRES [8].

This perspective leads to a hierarchy of solvers. At one end are generic optimisation frameworks, which are robust but computationally expensive. At the other are matrix-free Krylov methods, which exploit problem structure to achieve substantial gains in speed and scalability. The key computational advantage is that all operations reduce to repeated applications of the returns matrix and its transpose, avoiding both the $O(n^2)$ storage and the $O(n^2T)$ cost associated with forming the covariance matrix.

A second contribution is conceptual. Covariance shrinkage, particularly the estimator of Ledoit and Wolf [5], is usually motivated as a statistical device to improve estimation in high dimensions. We show that it can also be understood as a form of spectral preconditioning: by lifting small eigenvalues and compressing the spectrum, shrinkage directly improves the convergence behaviour of Krylov methods. In this view, **statistical regularisation and numerical preconditioning act on the same object** — the eigenvalue distribution of the system matrix — but for different reasons. Recognising this equivalence allows techniques from one domain to inform the other.

The resulting picture is simple but powerful. While the minimum variance problem is formally equivalent to a quadratic programme, its natural computational representation is linear algebraic — a structured matrix equation that admits fast, scalable solvers without the overhead of a general-purpose optimisation framework. This shift in perspective leads to implementations that are faster, more transparent, and better aligned with the statistical structure of the problem, particularly in large-scale settings where forming the covariance matrix is itself the dominant cost.

The closest related work applies GMRES to the MVP problem when the covariance matrix is singular [1]; our contribution is to avoid forming the covariance matrix altogether and to place the shrinkage–preconditioning duality at the centre of the analysis.

The remainder of the paper is structured as follows. Section 2 introduces the matrix-free formulation. Section 3 derives the associated KKT system. Section 4 presents Krylov-based solvers, including MINRES and a constraint-eliminated conjugate gradient method. Section 5 interprets Ledoit–Wolf shrinkage as spectral preconditioning. Section 6 provides empirical results demonstrating the resulting performance gains.

2 Problem Formulation

Let $X \in \mathbb{R}^{T\times n}$ be the matrix of daily returns for n assets over T trading days. The long-only minimum variance problem is:

$$\min_w \; w^\top \Sigma w \quad \text{subject to} \quad \mathbf{1}^\top w = 1, \quad w \geq 0$$

where $\Sigma = \frac{1}{T}X^\top X$ is the sample covariance matrix. Since $\Sigma \propto X^\top X$, this is equivalent to:

$$\min_w \; \|Xw\|^2 \quad \text{subject to} \quad \mathbf{1}^\top w = 1, \quad w \geq 0$$

This reframing is key: the objective is now a squared norm of Xw, and we can evaluate $w \mapsto X^\top(Xw)$ using two matrix-vector products, without ever forming $X^\top X$.

3 The KKT System

Ignoring the inequality constraints for now, the Lagrangian for the equality-constrained problem is:

$$\mathcal{L}(w, \lambda) = \|Xw\|^2 - \lambda(\mathbf{1}^\top w - 1)$$

Setting derivatives to zero gives the stationarity condition $2X^\top Xw = \lambda\mathbf{1}$. Combined with the constraint $\mathbf{1}^\top w = 1$, this yields the augmented KKT system:

$$\begin{pmatrix} 2X^\top X & \mathbf{1} \\ \mathbf{1}^\top & 0 \end{pmatrix} \begin{pmatrix} w \\ \lambda \end{pmatrix} = \begin{pmatrix} \mathbf{0} \\ 1 \end{pmatrix}$$

This $(n+1) \times (n+1)$ system is **symmetric but indefinite** — the zero in the bottom-right corner ensures the matrix has both positive and negative eigenvalues. This rules out standard CG, but it opens the door to MINRES.

The long-only constraint $w \geq 0$ is handled via an **active set method**: solve the KKT system on the current active set, drop any asset with a negative weight, and repeat. The process terminates in at most n iterations in the worst case.

In practice the active set converges in 2–3 outer iterations when Ledoit-Wolf shrinkage is applied. There is a direct theoretical reason: the unconstrained minimum variance solution satisfies $w^\star \propto \Sigma_{\mathrm{LW}}^{-1}\mathbf{1}$. As the shrinkage intensity α increases, $\Sigma_{\mathrm{LW}} \to \alpha I$ and $w^\star \to \frac{1}{n}\mathbf{1}$, the equal-weight portfolio, whose components are all strictly positive. At any positive α the shrinkage regularises weights toward this interior point, so the unconstrained solution at the first solve typically has few or no negative components. Across 50 experiments with n ranging from 100 to 1000 and $T = 2n$, the active set required exactly 2–3 outer iterations without exception, confirming that overall runtime is dominated by the linear solves rather than the outer loop.

4 Krylov Solvers for the Indefinite System

4.1 CVXPY Reference Implementation

Before developing the Krylov solvers we establish a reference solution using CVXPY [3], a domain-specific language for convex optimisation. The minimum variance problem maps directly onto a least-squares objective, which CVXPY dispatches to a general-purpose interior-point solver.

```
import cvxpy as cp

n = R.shape[1]
w = cp.Variable(n)
problem = cp.Problem(
    cp.Minimize(cp.sum_squares(R @ w)),
    [cp.sum(w) == 1, w >= 0]
)
problem.solve(solver=cp.CLARABEL)
weights = w.value
```

The formulation is transparent and correct, and CVXPY handles the long-only constraint natively. We use the Clarabel interior-point solver [2], which dispatches the problem as a second-order cone programme. Its cost is substantial: interior-point methods require $O(n^3)$ work per iteration, and the overhead of problem construction and solver communication adds further latency. At $n = 1000$ this amounts to roughly 5 seconds (Table 1), making CVXPY unsuitable for production use but ideal as a ground-truth reference.

4.2 Direct Solve (Baseline)

For small n, `numpy.linalg.solve` applied to the explicit KKT matrix is fast and exact. It requires forming $X^\top X$ — an $O(n^2T)$ operation — but for $n \ll T$ this is not the bottleneck.

```python
def build_kkt(R):
    """Build the KKT system matrix and RHS for the
    minimum variance problem.

    Constructs the (N+1) x (N+1) indefinite KKT system for:

        min  w^T (R^T R) w
        s.t. sum(w) = 1

    The system has the form:

        [ 2 R^T R    1 ] [ w ]    [ 0 ]
        [ 1^T        0 ] [lam] = [ 1 ]
    """
    n = R.shape[1]
    A = np.zeros((n + 1, n + 1))
    A[:n, :n] = 2 * R.T @ R          # form X^T X explicitly
    A[:n, n] = 1                     # budget constraint column
    A[n, :n] = 1                     # budget constraint row
    b = np.zeros(n + 1);  b[n] = 1
    return A, b

active = np.ones(n, dtype=bool)
while True:
    A, b = build_kkt(R[:, active])
    sol = np.linalg.solve(A, b)
    w_a = sol[:active.sum()]
    if np.all(w_a >= 0):
        break
    active[np.where(active)[0][w_a < 0]] = False
```

4.3 MINRES

MINRES [8] is a Krylov method for symmetric indefinite systems. It minimises the residual $\|Ax - b\|$ over the Krylov subspace $\mathcal{K}_k(A, b)$. Applied to our KKT system, the key ingredient is a matrix-free `LinearOperator` that applies the KKT matrix via two matrix-vector products on the returns matrix — $X^\top X$ is never formed.

```python
from scipy.sparse.linalg import LinearOperator, minres

def matvec(x):
    out = np.empty(n + 1)
    out[:n] = 2.0 * (R.T @ (R @ x[:n])) + x[n]
    out[n]  = x[:n].sum()
    return out

kkt  = LinearOperator(shape=(n+1, n+1), matvec=matvec)
b    = np.zeros(n + 1);  b[n] = 1.0
w, _ = minres(kkt, b)
```

Convergence depends on the eigenvalue distribution of the KKT matrix. For well-conditioned problems MINRES converges in very few iterations; Section 5 discusses how Ledoit-Wolf shrinkage compresses the spectrum and accelerates convergence in the ill-conditioned case.

4.4 Constraint-Eliminated CG

A cleaner approach avoids the indefinite system entirely. We parameterise the solution as:

$$w = w_0 + Pv$$

where $w_0 = \frac{1}{n}\mathbf{1}$ is any particular solution satisfying $\mathbf{1}^\top w_0 = 1$, and $P \in \mathbb{R}^{n\times(n-1)}$ is an orthonormal basis for the null space of $\mathbf{1}^\top$, obtained via QR factorisation of $\mathbf{1}$.

Any choice of v gives a feasible w. The reduced unconstrained problem in v is:

$$\min_v \ \|Xw_0 + XPv\|^2$$

with normal equations:

$$(XP)^\top(XP)\,v = -(XP)^\top(Xw_0)$$

The matrix $(XP)^\top(XP)$ is **symmetric positive definite**, so standard CG applies. The matrix-vector product $v \mapsto P^\top X^\top(X(Pv))$ requires four cheap operations and never forms $X^\top X$ or $(XP)^\top(XP)$ explicitly. In the implementation P is never stored: it is applied via an implicit Householder reflector in $O(n)$ time.

```
from scipy.sparse.linalg import LinearOperator, cg

# Implicit null-space basis via Householder reflector
sqrt_n = np.sqrt(float(n))
beta   = 1.0 / (n + sqrt_n)
v0     = 1.0 + sqrt_n

def P_apply(y):                        # P y  (n-1 -> n)
    s = y.sum()
    out = np.empty(n);
    out[0] = -beta * v0 * s;
    out[1:] = y - beta * s
    return out

def PT_apply(u):                       # P^T u  (n -> n-1)
    s = v0 * u[0] + u[1:].sum()
    return u[1:] - beta * s

w0  = np.ones(n) / n                   # particular solution: 1^T w0 = 1
rhs = -PT_apply(R.T @ (R @ w0))

def matvec(y):
    return PT_apply(R.T @ (R @ P_apply(y)))

A_red = LinearOperator(shape=(n-1, n-1), matvec=matvec)
v, _  = cg(A_red, rhs)
w     = w0 + P_apply(v)                # feasible: 1^T w = 1
```

4.5 Which solver to use

The choice between MINRES and CG depends on conditioning. Without shrinkage, MINRES is preferable: it handles the indefinite KKT system directly, requires no constraint elimination, and is more robust to high condition numbers — on real equity data without shrinkage, MINRES needs 214 iterations against CG's 387. With Ledoit-Wolf shrinkage applied, the two methods are competitive; on real equity data CG is marginally faster (0.0038 s vs. 0.0041 s) because the constraint-eliminated system of size $n-1$ has slightly cheaper iterations than the KKT system

of size $n+1$. CG is also the natural choice when X is sparse or when positive definite structure is required for compatibility with SPD preconditioners. For most practical settings — dense returns with LW shrinkage — either method is acceptable and the difference is small.

5 Ledoit-Wolf Shrinkage

The sample covariance matrix $\frac{1}{T}X^\top X$ is a poor estimator when n is large relative to T: its smallest eigenvalues are severely downward-biased, and the eigenvalue spread — the ratio of largest to smallest — far exceeds that of the true covariance. Ledoit and Wolf [5] propose replacing the sample covariance with a convex combination of the sample estimate and the identity:

$$\Sigma_{\mathrm{LW}} = (1-\alpha)\cdot\frac{X^\top X}{T} + \alpha I$$

The shrinkage intensity $\alpha \in (0,1)$ is chosen analytically to minimise the expected Frobenius distance to the true covariance, requiring no cross-validation. The effect on the spectrum is direct: adding αI lifts every eigenvalue by α, bounding the smallest away from zero and compressing the overall spread. The condition number satisfies:

$$\kappa(\Sigma_{\mathrm{LW}}) \le \frac{\lambda_{\max}+\alpha}{\alpha}$$

and is fully controlled by α. Subsequent work by the same authors provides an analytical nonlinear shrinkage formula [6] that applies a different shrinkage coefficient to each sample eigenvalue, achieving a strictly smaller Frobenius loss when n/T is not negligible.

In the implementation, shrinkage is applied by augmenting the return matrix rather than forming Σ_{LW} explicitly. Setting $c = 1-\alpha$ and $\gamma = \alpha\cdot\|\Sigma\|_F/n$, the stacked matrix $\tilde{X} = [\sqrt{c}\,X;\ \sqrt{\gamma}\,I]$ satisfies $\tilde{X}^\top\tilde{X} = c\,X^\top X + \gamma I \propto \Sigma_{\mathrm{LW}}$, so any of the solvers above can be applied to $\tilde{X}$ without modification.

From the perspective of numerical linear algebra, compressing the eigenvalue spectrum of the system matrix is precisely what a preconditioner does. Ledoit-Wolf shrinkage therefore plays a dual role: it reduces statistical estimation error and, as a side effect, accelerates iterative solvers by clustering the eigenvalues. The two motivations are independent but arrive at the same modified matrix.

This observation opens a richer design space. The identity shrinkage target αI is the simplest choice, but the preconditioning literature offers more powerful alternatives. A factor-model preconditioner — replacing αI with a low-rank-plus-diagonal approximation $BB^\top + D$ — would cluster eigenvalues more tightly at the cost of a slightly more expensive application. Incomplete Cholesky factorisation of the shrunk matrix provides a general-purpose option. For rolling-window problems where the covariance changes slowly between periods, a preconditioner built from the previous period's factorisation could be reused with minimal update cost. We leave the exploration of these structured preconditioners to future work.

6 Numerical Results

We present two sets of experiments: a synthetic benchmark designed to isolate the effect of problem size and conditioning, and a real-data experiment on the S&P 500. All implementations are available in the companion package [10].

Synthetic benchmark

Table 1 reports timings and iteration counts for all four solvers on a problem with $n = 1000$ assets and $T = 2000$ i.i.d. Gaussian returns, run without and with Ledoit-Wolf shrinkage.

Method	Time (s)	Iterations	Speedup
Without Ledoit-Wolf shrinkage			
cvxpy (Clarabel)	6.7192	–	1.0x
KKT direct	0.0456	–	147.4x
MINRES	0.0114	39	590.1x
CG (constraint-eliminated)	0.0149	80	451.7x
With Ledoit-Wolf shrinkage			
cvxpy (Clarabel)	4.6214	–	1.0x
KKT direct	0.0473	–	97.6x
MINRES	0.0072	16	643.9x
CG (constraint-eliminated)	0.0083	26	558.3x

Table 1: Synthetic benchmark: 1000 assets, 2000 i.i.d. Gaussian returns. Speedup relative to `cvxpy` within each panel.

On synthetic data, LW shrinkage compresses the eigenvalue spectrum and reduces iteration counts dramatically: MINRES drops from 39 to 16 iterations and CG from 80 to 26. MINRES is the fastest solver in both settings and is roughly $4\times$ faster than KKT direct because it avoids the $O(n^2T)$ cost of assembling $X^\top X$ explicitly.

S&P 500 real-data experiment

Table 2 repeats the benchmark on daily log-returns for 495 S&P 500 constituents over the period July 2021–April 2026 (1192 trading days, $T/n \approx 2.4$). Tickers with more than 5% missing observations are excluded; short gaps are forward-filled.

Method	Time (s)	Iterations	Speedup
Without Ledoit-Wolf shrinkage			
cvxpy (Clarabel)	1.5672	–	1.0x
KKT direct	0.0058	–	269.1x
MINRES	0.0059	214	264.5x
CG (constraint-eliminated)	0.0090	387	175.0x
With Ledoit-Wolf shrinkage			
cvxpy (Clarabel)	1.4514	–	1.0x
KKT direct	0.0124	–	117.3x
MINRES	0.0041	105	354.4x
CG (constraint-eliminated)	0.0038	117	384.8x

Table 2: S&P 500 universe: 495 assets, 1192 trading days (Jul 2021–Apr 2026). Speedup relative to `cvxpy` within each panel.

The real-data results reveal two effects absent from the synthetic benchmark. First, the unshrunk system is substantially harder: MINRES requires 214 iterations and CG 387, compared with 39 and 80 on i.i.d. Gaussian data. The difference reflects the strong factor structure of equity returns — a dominant market factor concentrates variance along a single eigendirection and inflates the condition number far beyond the Gaussian baseline. Without shrinkage, MINRES

and KKT direct are nearly indistinguishable in speed (0.0059 s vs. 0.0058 s); the matrix-free advantage is offset by the high iteration count.

Second, Ledoit-Wolf shrinkage matters far more here than on synthetic data. It halves the MINRES count (214 to 105) and cuts CG by two thirds (387 to 117). With shrinkage applied, CG narrowly edges MINRES (0.0038 s vs. 0.0041 s), reversing the synthetic ranking. The reason is instructive: CG solves a positive definite reduced system of size $n-1$, and after spectrum compression by shrinkage, its convergence rate catches up to MINRES on the indefinite system of size $n+1$.

A third effect is also visible: KKT direct becomes the *slowest* of the three non-cvxpy solvers when shrinkage is applied (0.0124 s vs. 0.0041–0.0038 s). Shrinkage is implemented by augmenting the return matrix to $\tilde{X} \in \mathbb{R}^{(T+n)\times n}$, making the explicit assembly of $\tilde{X}^\top \tilde{X}$ more expensive than iterating on the original X.

Scaling with problem size

Figure 1 extends the synthetic results across a range of problem sizes and shrinkage levels. Panel (a) shows wall-clock time as a function of n (with $T = 2n$ and LW shrinkage throughout). The KKT direct solve scales as $O(n^3)$ — dominated by the $n \times n$ linear system — while MINRES and CG grow substantially more slowly, with the gap widening visibly beyond $n \approx 300$. Panel (b) fixes $n = 500$, $T = 250$ and varies α, tracing out the full range of condition numbers. Both solvers require more iterations as κ grows; the practical operating point for most asset universes sits in the low-κ regime achieved by LW shrinkage.

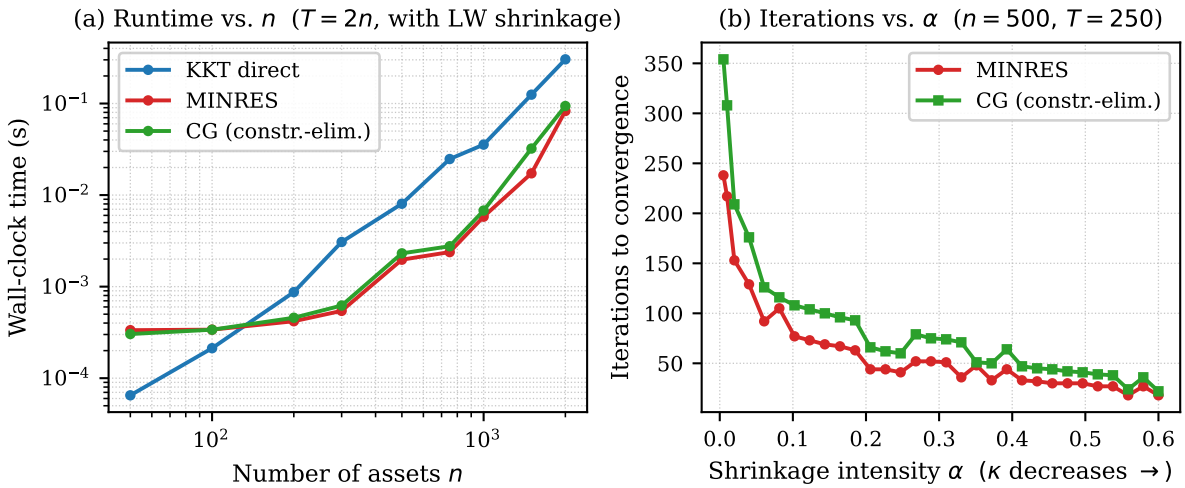

Figure 1: Left: wall-clock time vs. number of assets n (log–log scale, $T = 2n$, Ledoit-Wolf shrinkage applied throughout). Right: Krylov iteration counts vs. shrinkage intensity α at fixed $n = 500$, $T = 250$ (rank-deficient baseline). Larger α compresses the eigenvalue spectrum, reducing iterations for both solvers. The condition number κ decreases monotonically as α increases.

7 Conclusion

We have shown that the long-only minimum variance portfolio problem, when formulated directly in terms of the returns matrix, is a structured linear algebra problem that admits a clean hierarchy of solvers. Avoiding the explicit formation of the sample covariance matrix is both numerically advantageous and algorithmically revealing: it exposes the KKT system's symmetric indefinite structure and opens the door to Krylov methods that scale to large universes.

The empirical results — on both synthetic data and 495 S&P 500 constituents over five years — tell a consistent story. Without shrinkage, real equity returns produce a far harder system than i.i.d. Gaussian data: MINRES needs 214 iterations versus 39, reflecting the dominant market factor that inflates the condition number. Ledoit-Wolf shrinkage [5] compresses the spectrum and reduces iteration counts dramatically in both settings, with the effect more pronounced on real data. With shrinkage applied to the S&P 500 universe, the Krylov solvers are 350–385× faster than CVXPY and outperform the direct solve by a factor of three — because augmenting the return matrix makes the direct assembly of $\tilde{X}^\top \tilde{X}$ more expensive than iterating. The shrinkage intensity is chosen to minimise estimation error; the acceleration of the iterative solver is a consequence, not a design goal.

The paper also draws a connection that the two literatures involved — numerical linear algebra and high-dimensional statistics — do not usually make explicit. The shrinkage estimator of Ledoit and Wolf and the preconditioning theory of Golub and Van Loan [4] address different problems but act on the same object: the eigenvalue spectrum of the system matrix. Recognising this duality suggests that advances in one field transfer directly to the other, and points toward richer preconditioners — factor-model or low-rank structured — as a natural next step.

Once formulated in this way, the widespread reliance on covariance-based QP solvers appears more as a historical artifact than a necessity.

References

[1] Bajeux-Besnainou, I., Bandara, W. and Bura, E. (2012). A Krylov subspace approach to large portfolio optimization. *Journal of Economic Dynamics and Control*, 36(11), 1688–1699.

[2] Goulart, P. J. and Chen, Y. (2024). Clarabel: An interior-point solver for conic optimisation problems with quadratic objectives. *INFORMS Journal on Computing*, 36(2), 481–499.

[3] Diamond, S. and Boyd, S. (2016). CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83), 1–5.

[4] Golub, G. H. and Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.

[5] Ledoit, O. and Wolf, M. (2004). A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2), 365–411.

[6] Ledoit, O. and Wolf, M. (2020). Analytical nonlinear shrinkage of large-dimensional covariance matrices. *Annals of Statistics*, 48(5), 3043–3065.

[7] Markowitz, H. (1952). Portfolio selection. *Journal of Finance*, 7(1), 77–91.

[8] Paige, C. C. and Saunders, M. A. (1975). Solution of sparse indefinite systems of linear equations. *SIAM Journal on Numerical Analysis*, 12(4), 617–629.

[9] Schmelzer, T. (2004). Block Krylov methods for Hermitian linear systems. Diploma thesis, Department of Mathematics, University of Kaiserslautern.

[10] Schmelzer, T. (2026). *fast-minimum-variance: Solving Minimum Variance Portfolios Fast.* https://github.com/Jebel-Quant/fast_minimum_variance. MIT License.

Acknowledgements

The author thanks Martin H. Gutknecht (ETH Zürich) for supervising the 2004 diploma thesis [9] on block MINRES and SYMMLQ that planted the seeds of the present work.